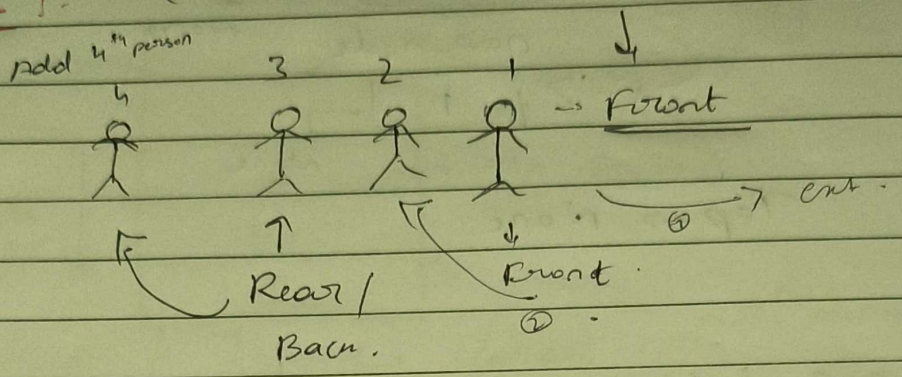
25/8/25
Day 33

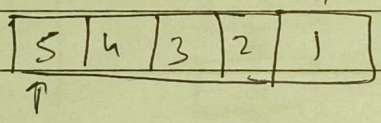
13 Introduction to Queue

Queue : (FIFO)



When Front = rear } Comes on same index
Called Empty Queue.

front del / remove



rear

capacity = 5.
full

always. Insertion / Add

→ Rear :- Adding / Insertion (Enqueue)

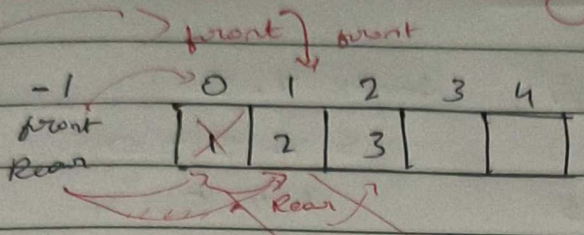
→ Front :- Removing / Del (Dequeue)

→ capacity = max.

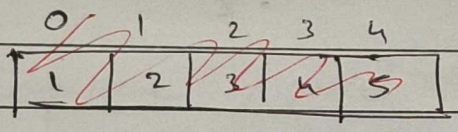
→ Front == Rear ⇒ Empty.

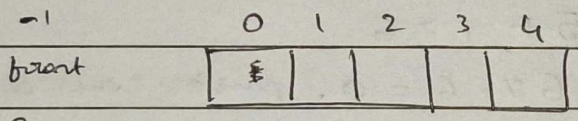
Peek → Value.

Queue Implementation Using Array :

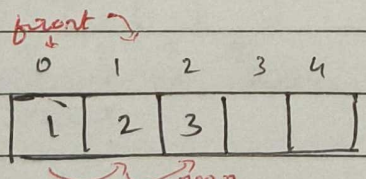


enqueue (1)
enqueue (2)
enqueue (3)
dequeue ()

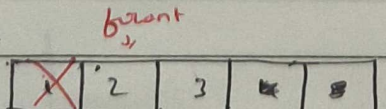




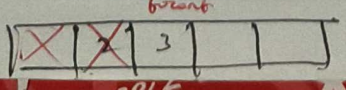
→ enqueue (1)
enqueue (2)
enqueue (3)



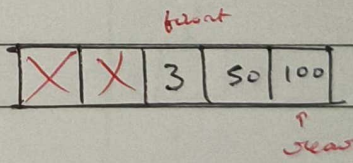
dequeue ()



dequeue ()



enqueue (50)
enqueue (100).



capacity = 5 max - 1 = 4

enque (200) -> X fail

When all delete & front & rear come to same index then it will insert from 1st but this the drawback otherwise not insert.

Modulus %. Operator:-

modules of 5. index %. 5.

0 → 1 → 2 → 3 → 4 → 5

5 %. 5 = 0 → remainder.

4 %. 5 = 4

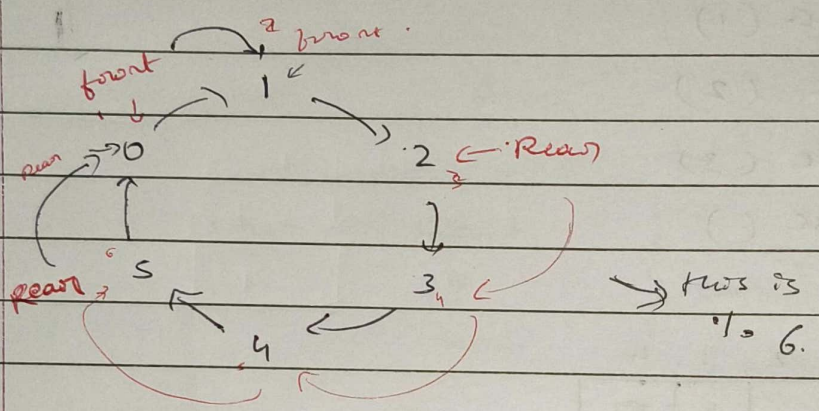
3 %. 5 = 3

1 %. 5 = 1

0 %. 5 = 0.

10 %. 5 = 0

11 %. 5 = 1.



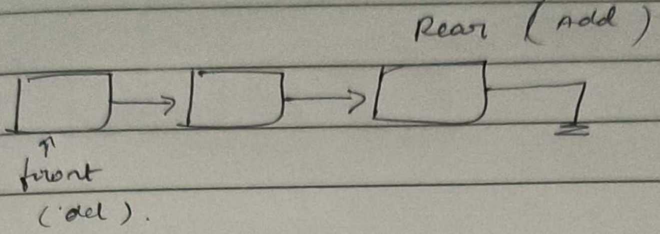
→ when del one

→ add one. 5 + 1 = 6

6 %. 6 = 0. push the 0. after 5.

Singly linked list in Queue :-

DMA -> heap memory



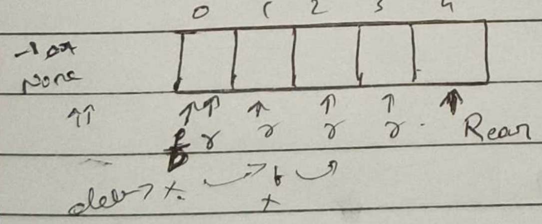
B Queue in Python :- (FIFO)

-> Enqueue / Add

-> Dequeue / Remove

-> Peek ()
-> count()
-> Print() ? Readymade / Built in fun

#1 -> Array / List



enqueue O(1)

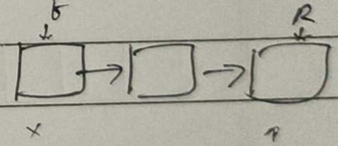
deque O(n)

#2 Collection :

dequeue :

#3 Custom Impl^n

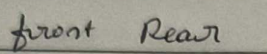
using Singly linked list



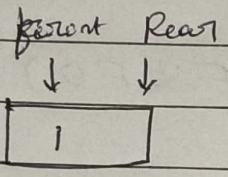
O(1)

O(1)

Using SLL :-

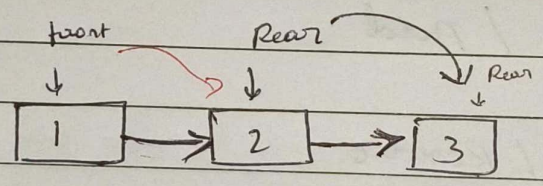


Adding 1

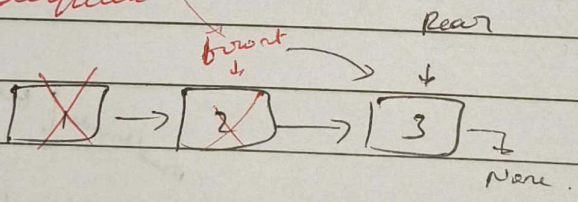


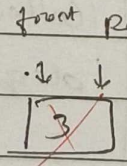
Enqueue / Insert:

Adding 2/3



for dequeue





front Rear

None.

creating the queue.py > ...

```python
class Node:
    def __init__(self,data):
        self.data = data
        self.next =  None

class Queue:

    def __init__(self):

        self.rear = None
        self.front = None
        self.count = 0

    def enqueue(self,data):
        new_node = Node(data)

            # queue is empty                    # front → [1] → [2] → [3] → None   : front is stay 1st only
                                                # rear ------------^                only rear is trverses

        if self.rear is None:
            self.front = new_node
            # self.rear = new_node

        else:
            self.rear.next = new_node
            # self.rear = new_node

        self.rear = new_node
        self.count += 1
        print(f"element {data} is inserted in queue")

    def dequeue(self):
        # queue is empty
        if self.front is None:
            print("queue is empty")
            return -100
```

```
6   class Queue:
32      def dequeue(self):
37
38          return_data = self.front.data
39          self.front = self.front.next
40
41          if self.front is None:
42              self.rear = None
43
44          self.count -= 1
45          print(f"Removing element {return_data} from the queue")
46          return return_data
47
48      def peek(self) -> int:
49
50          if self.front is None:
51              print("queue is empty")
52              return -100
53
54          print("peek is", self.front.data)
55          return self.front.data
56
57      def get_count(self) -> int:
58          print("count is ",self.count)
59          return self.count
60
61      def print_all_nodes(self):
62
63          if self.front is None:
64              print("queue is empty")
65              return -100
66
67          print("the queue elements are ")
68          current_node = self.front
69
70          while current_node is not None:
71              print(current_node.data, end = " -> ")
```

```
69
70          while current_node is not None:
71              print(current_node.data, end = " -> ")
72              current_node = current_node.next
73
74          print("None")
75
76
77  que = Queue()
78  que.dequeue()
79  que.print_all_nodes()
80
81  que.enqueue(1)
82  que.print_all_nodes()
83  que.dequeue()
84  que.print_all_nodes()
85
86  que.enqueue(1)
87  que.enqueue(2)
88  que.enqueue(3)
89
90  que.print_all_nodes()
91  que.get_count()
92  que.peek()
93  que.dequeue()
94  que.print_all_nodes()
95
96
97
98
```